

CSCE 671 HUMAN-COMPUTER INTERACTION · MODULE 06 · LECTURE NOTES

Prototyping

Building the cheapest thing that answers the question.

MODULE OUTCOMES

- Explain why low fidelity gets better feedback.
- Choose fidelity from the question being asked.
- Explain Wizard of Oz and when it is the right method.
- Explain what prototypes cannot test.
- Prototype at the right level for a stated question.

1 Why low fidelity

PEOPLE CRITIQUE A SKETCH AND DEFER TO A POLISHED MOCK-UP

A hand-drawn prototype invites criticism — it visibly cost nothing to produce, so saying it is wrong costs the reviewer nothing either, socially or otherwise.

A polished one invites approval. It visibly took considerable effort, and the feedback shifts toward colours, wording, and alignment and away from whether the whole approach is right — which is precisely the feedback you needed and the only kind that is still cheap to act on.

And you are more willing to discard a sketch, which matters because **the sunk cost of a built prototype distorts your own judgement** at least as much as the reviewer's — and yours is the judgement that decides.

So the fidelity is a choice about what feedback you will get — not a budget constraint — and low fidelity early is a method rather than a compromise, which is the reframing this module asks for.

2 Choosing fidelity

VISUAL FIDELITY how finished it looks
 FUNCTIONAL BREADTH how many features exist at all
 FUNCTIONAL DEPTH how completely any one of them
 works

AND YOU VARY THEM INDEPENDENTLY, which is the point:

"Is the conceptual model right?"
 -> low visual, broad, no depth. A paper
 walkthrough of every screen, nothing working.

"Can people complete this task?"
 -> low visual, narrow, full depth. One path,
 genuinely working, and ugly.

"Does it feel responsive enough?"
 -> high visual, narrow, full depth, with real
 timing. Nothing else matters for this
 question.

"Will people want it?"
 -> a prototype cannot answer this (section 4).

SO: name the question first, then build the least
 thing that answers it.

THE COMMONEST PROTOTYPING ERROR IS UNIFORM FIDELITY

A prototype built to a uniform level of polish spends effort on the parts that are not in question — which is most of the effort, and it buys nothing except a longer delay before you learn anything.

So deliberately leave things broken. A prototype with one working path and nine dead links tests the path perfectly well, and the dead links cost nothing provided the participant is told in advance which parts are not built.

And conversely, do not under-build the thing that is in question. A latency question needs real latency; a typography question needs real type; a density question needs real content — and a sketch cannot answer any of them, so a sketch is the wrong tool there.

Which is why the question has to come first — a prototype built before the question is named will be uniformly polished and will answer nothing in particular, which describes a great many prototypes and explains why their reviews are unsatisfying.

3 Wizard of Oz

A PERSON BEHIND THE CURTAIN LETS YOU TEST AN INTERACTION BEFORE THE SYSTEM EXISTS

A human performs the system's function while the participant believes it is automated — which tests the *interaction* without requiring the *capability* to exist, and is the only method in this course that can do that.

Which makes it uniquely valuable for anything expensive to build: a speech interface, a recommendation system, a generated response, or an automation — all of which cost a great deal and may turn out to be the wrong design, which is an expensive thing to discover afterwards.

And it answers questions the built system cannot: what do people actually say to it, what do they expect it to handle, and how do they recover when it fails? — all of which you need before choosing what to build, and none of which you can find out from a specification.

With one ethical requirement: debrief afterwards (Module 08 §4's ethics) — the deception is methodologically necessary and has to be undone, explicitly and before the participant leaves.

- Decide the wizard's rules in advance, and follow them — because a wizard who is cleverer than the real system could ever be gives you a misleading result, and the temptation to be helpful is strong when a participant is struggling.
- Include failures deliberately, at approximately the rate the real system would have — since how people recover from a failure is frequently the actual design question, and a flawless wizard tests nothing about it.
- Record the inputs, because what people actually say or type is the most valuable output of the whole exercise — it is your training and test data later, and it is unobtainable any other way.
- Keep the latency realistic, since response time changes behaviour substantially — a wizard who responds instantly produces interaction patterns the real system will not see.
- And debrief. Always. 'What people actually say to it' is the finding, and it is routinely quite different from what the designers assumed — which is the whole reason to run the study and the thing to write down first.

4 Limits

- Whether people want it. A participant in a room will engage politely with almost anything, and expressed interest predicts actual adoption poorly — which is a robust finding and a persistent temptation.
- Long-term behaviour. Learnability over weeks, habituation, and whether the novelty wears off all require time that a single prototype session does not have, and all of them can reverse a short-term result.
- Performance at scale, and anything depending on real data volume, real content quality, or the presence of real other users — a prototype's clean sample data hides a great deal.

- **And social dynamics** — how a feature behaves when other people can see what you did with it, which changes behaviour fundamentally and cannot be simulated in a room.
- **So state what your prototype tested and what it could not**, which is Module 13 §2's requirement. **Expressed interest predicts adoption poorly** — which is why **'would you use this' is nearly worthless as a question** and **'show me how you do this now' is not**: the second asks about behaviour that already exists.

AND THE DISCIPLINE THIS MODULE ASKS FOR

Write the question before you build anything, in a single sentence — and **if you cannot write it, you are not ready to prototype**, which is a useful and uncomfortable test.

Build the least thing that answers it, varying the three dimensions independently (§2) and **deliberately leaving the rest broken**.

Then test it with people (Module 07) — **because a prototype nobody used is a drawing**, and a drawing you evaluated yourself is subject to Module 01 §2 in full.

And throw it away. A prototype that becomes the implementation carries every shortcut you took deliberately — the hardcoded values, the missing error handling, the one working path — **which is a real and very common cost, and is worth naming before it happens** rather than discovering it two years later (CSCE 606's technical debt, acquired on purpose).

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Buxton — Sketching User Experiences	§1's argument at length — why sketching and prototyping are different activities. Library copy.
Snyder — Paper Prototyping	§1 and §2 practically — how to run a paper prototype session, in detail. Library copy.
Dahlbäck, Jönsson & Ahrenberg — Wizard of Oz studies (free)	§3's method and its pitfalls — including the wizard-is-too-clever problem.
Nielsen Norman Group on prototyping fidelity (free)	§2's decision, practically — and the independent dimensions made explicit.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Sketch an interface on paper** and show it to three people.
- 2 **Build the same thing polished** and show it to three others.
- 3 **Compare the feedback** and classify it as structural or cosmetic.
- 4 **Write four design questions** and specify the fidelity each needs on all three dimensions.
- 5 **Build one of them** and deliberately leave the rest broken.
- 6 **Run a Wizard of Oz study** for an interaction you have not built.
- 7 **Record every input** and compare against what you expected people to say.
- 8 **Include deliberate failures** and observe the recovery.
- 9 **Ask 'would you use this' and 'show me how you do this now'**, and compare what you learn.
- 10 **List what your prototype could not test.**

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why do people critique a sketch and defer to a mock-up?
- 2 Why is fidelity a choice about feedback rather than budget?
- 3 Name the three dimensions of fidelity.
- 4 Give the right fidelity for three different questions.
- 5 What is the commonest prototyping error, and what prevents it?
- 6 What does Wizard of Oz test that a built system cannot?
- 7 Give five rules for running one, and the ethical requirement.
- 8 Name four things a prototype cannot answer.
- 9 Why is 'would you use this' nearly worthless?
- 10 Why throw the prototype away?